

Ejercicio práctico III

Teniendo en cuenta el código que se muestra a continuación, y que figura en el último apartado del tema de “Multiprocesadores de Memoria Compartida”:

```
int *array;
int length, count;

int count3s()
{
    int i;
    count = 0;
    for (i=0; i<length; i++)
    {
        if (array[i] == 3)
            count++;
    }
    return count;
}
```

- a) Realizar la programación de dicho código, que cuenta el número de veces que aparece el número 3 en un vector, midiendo el **tiempo de ejecución** del mismo para un vector de números enteros de, al menos, 100 millones de elementos generados de forma aleatoria entre 0 y 100.
- b) Implementar una versión paralela de dicho código con Pthreads, incorporando sección crítica. Similar al código mostrado en el mencionado documento y que se indica a continuación, obteniendo también el tiempo de ejecución:

```
int t;      // numero de threads
int *array, length, count;

int count3s()
{
    int i; count = 0;
    for (i=0; i<t; i++)    // crear t threads
        thread_create(count3s_thread, i);
    return count;
}
```

```

mutex m;

int count3s_thread(int id)
{
    int i; int length_per_thread = length/t;
    int start = id*length_per_thread;
    for (i=start; i<start+length_per_thread; i++)
        if (array[i] == 3)
        {
            mutex_lock(m);
            count++;
            mutex_unlock(m);
        }
}

```

- c) Realizar la implementación paralela mediante Pthreads contemplando la estructura “paddeada” para cada contador de hilo tal y como figura al final del citado documento y que se muestra a continuación, obteniendo los tiempos de ejecución.

```

int t;      // numero de threads
int *array, length, count;

int count3s()
{
    int i; count = 0;
    for (i=0; i<t; i++)    // crear t threads
        thread_create(count3s_thread, i);
    return count;
}

mutex m;
struct padded_int
{
    int value;
    char padding[60];
} private_count[MaxThreads];

int count3s_thread(int id)
{
    int i; int length_per_thread = length/t;
    int start = id*length_per_thread;
    for (i=start; i<start+length_per_thread; i++)
        if (array[i] == 3)
            private_count[id].value++;
    mutex_lock(m);
    count += private_count[id].value;
    mutex_unlock(m);
}

```

- d) Para cada una de las implementaciones paralelas anteriores (b y c): Comparar las ejecuciones para distinto número de hilos (2, 4, 8, 16, 32), obteniendo el speed-up y eficiencia y realizando una gráfica de las mismas.
- e) Justificar los resultados los obtenidos. Realizando una comparación de la ejecución secuencial con cada uno de los mejores resultados de las versiones paralelas.
- f) Implementar, mediante OpenMP, una versión paralela del código desarrollado en el apartado a) para distinto número de hilos, obteniendo el speed-up. Justificar resultados y desarrollar conclusiones.

Nota: Para medir los tiempos tanto del programa secuencial como de los programas paralelos pthread, utilizar la función "omp_get_wtime()". Recordar incluir la librería omp.h y compilar con el flag -fopenmp.